

Experiment 7

Student Name: Nikhil Kumar

UID: 25MCI10036

Branch: MCA (AI/ML)

Section/Group: 25MAM-1(A)

Semester: 2-SEM

Date of Performance: 31/03/2026

Subject Name: Technical Training - 1

Subject Code: 25CAP-652

1. Aim of the Session:

Implementation of joins in PostgreSQL (inner join, left join, right join, self-join and cross join)

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

4. Theory:

A **JOIN** in SQL is used to combine rows from two or more tables based on a related column between them. Joins are essential in relational databases because data is often stored in multiple tables to reduce redundancy and improve organization.

Types of Joins Used in this Experiment:

- **INNER JOIN:** Returns only the matching records from both tables.
- **LEFT JOIN:** Returns all records from the left table and the matching records from the right table. If no match is found, NULL values are returned for the right table columns.
- **RIGHT JOIN:** Returns all records from the right table and the matching records from the left table. If no match is found, NULL values are returned for the left table columns.
- **SELF JOIN:** A table is joined with itself to represent relationships within the same table, such as mentor-student or manager-employee relationships.
- **CROSS JOIN:** Returns the Cartesian product of two tables, i.e., all possible combinations of rows from both tables.

5. Practical / Experiment Steps:

Step 0: Creating sample tables and inserting records

Code:

```
Create table students(  
stud_id int primary key,  
name varchar(50),  
dept_id int  
);
```

```
create table courses(  
course_id int primary key,  
course_name varchar(50)  
);
```

```
create table enrollments(  
stud_id int,  
course_id int,  
primary key(stud_id, course_id),  
foreign key(stud_id) references students(stud_id),  
foreign key(course_id) references courses(course_id)  
);
```

```
create table departments(  
dept_id int primary key,  
dept_name varchar(50)  
);
```

```
insert into students values  
(1,'Nikhil',101),  
(2,'Aman',102),  
(3,'Riya',101),  
(4,'Sanya',103),  
(5,'Rahul',Null);
```

```
insert into courses values  
(201,'DBMS'),  
(202,'OS'),  
(203,'CN'),  
(204,'AI');
```

```
insert into enrollments values
```

(1,201),
(1,202),
(2,201),
(3,203);

insert into departments values
(101,'Computer science'),
(102,'Electronics'),
(103,'Mechanical');

Step 1: List all students with their enrolled courses (INNER JOIN).

Code:

```
select s.name,c.course_name  
from students s  
inner join enrollments e on s.stud_id=e.stud_id  
inner join courses c on e.course_id=c.course_id;
```

Output:

	name character varying (50) 🔒	course_name character varying (50) 🔒
1	Nikhil	DBMS
2	Nikhil	OS
3	Aman	DBMS
4	Riya	CN

Step 2: Find students not enrolled in any course (LEFT JOIN).

Code:

```
select s.name from students s  
left join enrollments e on s.stud_id=e.stud_id  
where e.stud_id is Null;
```

Output:

	name character varying (50) 🔒
1	Rahul
2	Sanya

Step 3: Display all courses with or without enrolled students (RIGHT JOIN).

Code:

```
select c.course_name,s.name
from students s
right join enrollments e on s.stud_id=e.stud_id
right join courses c on e.course_id=c.course_id;
```

Output:

	course_name character varying (50) 🔒	name character varying (50) 🔒
1	DBMS	Nikhil
2	OS	Nikhil
3	DBMS	Aman
4	CN	Riya
5	AI	[null]

Step 4: Show students with department info using SELF JOIN or multiple joins.

Code:

```
select s.name,d.dept_name
from students s
left join departments d
on s.dept_id=d.dept_id;
```

Output:

	name character varying (50) 🔒	dept_name character varying (50) 🔒
1	Nikhil	Computer science
2	Aman	Electronics
3	Riya	Computer science
4	Sanya	Mechanical
5	Rahul	[null]

Step 5: Display all possible student-course combinations (CROSS JOIN).

Code:

```
select s.name,c.course_name
from students s
```

cross join courses c;

Output:

	name character varying (50)	course_name character varying (50)
1	Nikhil	DBMS
2	Aman	DBMS
3	Riya	DBMS
4	Sanya	DBMS
5	Rahul	DBMS
6	Nikhil	OS
7	Aman	OS
8	Riya	OS
9	Sanya	OS
10	Rahul	OS
11	Nikhil	CN
12	Aman	CN
13	Riya	CN
14	Sanya	CN
15	Rahul	CN
16	Nikhil	AI
17	Aman	AI
18	Riya	AI
19	Sanya	AI
20	Rahul	AI

6. Learning Outcomes (What I have learned):

- **Join Understanding:** I learned how to use different types of SQL joins to retrieve related data from multiple tables.
- **Practical Database Skills:** I understood how real-world schemas such as Students, Courses, Enrollments, and Departments are related in a relational database.
- **Data Analysis Ability:** I learned how to identify matching and non-matching records using INNER JOIN, LEFT JOIN, and RIGHT JOIN.
- **Advanced SQL Usage:** I understood the use of SELF JOIN for hierarchical relationships and CROSS JOIN for generating all possible combinations.